Course: AI-Based Programming

Week 2 Lecture Notes: Data Engineering for AI Systems

Subtitle: Building the Backbone of Reliable Machine Learning

1. Introduction: The Foundation of AI Systems

Last week, we discussed the transition from writing isolated machine learning scripts to building full-scale AI software systems. Today, we are focusing on the absolute foundation of those systems: **Data Engineering**.

Without robust data pipelines, even the most advanced AI models will fail in production. Today we will cover how to ingest, scale, validate, format, and version our data to ensure our AI systems are reliable, scalable, and reproducible.

2. The "Hidden Technical Debt" of Machine Learning

There is a common myth in the industry that AI systems are primarily composed of machine learning code (neural networks, optimization algorithms, etc.).

The Reality: ML code makes up only about **5%** of a real-world AI system. The vast majority of the system involves infrastructure, serving infrastructure, and crucially, **data pipelines**.

Why Data Engineering Matters:

- **Prevents "Garbage In, Garbage Out" (GIGO):** The best model cannot fix bad data.
- **Ensures Reliability:** Models perform consistently over time only if the data feeding them is consistent.
- **Enables Scalability:** Proper engineering allows a system to scale from processing megabytes on a local laptop to terabytes in the cloud seamlessly.

Instructor Note: Refer to the famous "Hidden Technical Debt in Machine Learning Systems" diagram by Google. Emphasize that if you don't engineer your data correctly, the sophisticated neural network in the middle is rendered completely useless.

3. Data Ingestion: Batch vs. Stream Processing

Definition: Data Ingestion is the process of extracting, transporting, and loading data from various external sources (like databases, APIs, IoT sensors, or user clickstreams) into a centralized storage medium where it can be accessed, used, and analyzed by the system.

The architecture you choose for ingestion fundamentally dictates what your AI can achieve:

- **Batch Processing:** * *Concept:* Processes large volumes of data at scheduled intervals (e.g., nightly, weekly).
    - *Tools:* Apache Airflow, AWS Glue.
    - *Use Cases:* Training large foundation models, updating daily recommendation engines, or calculating nightly aggregate metrics.
- **Stream Processing:**
    - *Concept:* Processes data continuously in near real-time as it is generated.
    - *Tools:* Apache Kafka, AWS Kinesis.
    - *Use Cases:* Fraud detection at the point of sale, live autonomous vehicle perception, real-time dynamic pricing.

4. Modern Data Pipelines: ETL vs. ELT & Storage Architectures

Historically, data systems used the **ETL** paradigm. Today, AI systems heavily favor **ELT**.

- **ETL (Extract, Transform, Load):** Data is transformed *before* it reaches the central storage. This is still useful for environments with strict privacy rules where data masking must happen immediately before hitting central storage.
- **ELT (Extract, Load, Transform):** Raw data is loaded directly into a central storage system, and cloud compute is used to transform it later.

Key Infrastructure Definitions: Data Lake vs. Data Warehouse To understand where we load this data during ELT, we must define our core storage architectures:

- **Data Warehouse:** A highly structured repository for filtered, processed data that has already been transformed for a specific purpose (schema-on-write). It typically stores tabular, relational data and is optimized for fast business intelligence querying.
    - *Analogy:* Bottled, purified water ready to drink.
- **Data Lake:** A vast pool of raw, unformatted data whose ultimate purpose may not yet be defined (schema-on-read). It can store structured, semi-structured (JSON, XML), and entirely unstructured data (images, audio, text logs).
    - *Analogy:* A natural body of water in its raw state.

The AI Advantage of ELT & Data Lakes: Compute in modern cloud environments has become incredibly cheap and powerful. ELT keeps the **raw data intact** inside a Data Lake. In machine learning, we never want to permanently alter or lose our raw data. If the AI team discovers they need to extract a brand new feature from the data 6 months later, the original, unmutated data is still there waiting for them in the lake.

5. Data Preprocessing & Feature Scaling

AI models—especially neural networks and distance-based algorithms (like KNN or SVMs)—require data to be on a standardized scale. They do not intuitively understand that a house price of $500,000 and a bedroom count of 3 belong to vastly different conceptual scales. Without scaling, the model's weights will struggle to converge during gradient descent optimization.

Method 1: Min-Max Scaling (Normalization)

Binds values strictly within a specific range, typically between 0 and 1. Use this when you need strict mathematical bounds.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Numerical Example:** Imagine a dataset of house prices where the minimum price in the dataset is **$200,000** and the maximum is **$1,000,000**. We want to scale a house priced at **$500,000**.

$$X_{norm} = \frac{500000 - 200000}{1000000 - 200000} = \frac{300000}{800000} = 0.375$$

(The $500k house translates smoothly into a machine-friendly 0.375)

Method 2: Z-Score Standardization

Centers the data around a mean (μ) of 0 with a standard deviation (σ) of 1. Use this when your data has extreme outliers that you do not want to "squish" into a tiny 0-1 boundary.

$$Z = \frac{X - \mu}{\sigma}$$

6. Preprocessing: The Danger of Outliers in Imputation

Handling missing data is a critical pipeline step, but choosing the right statistical metric matters mathematically and can directly impact model accuracy.

Numerical Example: Imputing missing salaries. Consider a company dataset of 5 employees: $40k, $45k, $50k,

$$MISSING$$

, $1,000k (CEO)

- Option A (Mean Imputation - Dangerous here):
  - Sum of knowns: $40 + 45 + 50 + 1000 = 1135$
  - Mean calculation: $1135/4 = 283.75$
  - *Result:* We impute **$283.75k** for the missing standard employee. This is heavily skewed by the CEO outlier and does not represent reality.
- Option B (Median Imputation - Robust):
  - Sorted knowns: $40, 45, 50, 1000$
  - Median: The midpoint between 45 and 50 is **$47.5k**.
  - *Result:* Much closer to the standard employee distribution.

7. Silent Failures & Automated Data Validation

In traditional software engineering, bad inputs usually cause a system to crash (e.g., a NullPointerException). In AI, bad inputs cause the model to **silently output bad predictions**.

Imagine an upstream sensor breaks and starts sending negative values for human age or income. Your Python code will not crash; it will happily feed that to the neural network, which will confidently output garbage.

The Solution: Data Contracts & Automated Validation using tools like **Pydantic** (Python).

```
from pydantic import BaseModel, Field, ValidationError

# Define the exact contract your data must follow before hitting the AI model
class UserDataInput(BaseModel):
    age: int = Field(..., ge=0, le=120)  # Age strictly bounded 0-120
    income: float = Field(..., ge=0.0)   # Income cannot be negative
    user_segment: str

# Example Pipeline Check:
try:
    # Simulating a broken upstream sensor sending bad data
    bad_data = UserDataInput(age=150, income=-500, user_segment="A")
except ValidationError as e:
    print("Pipeline HALTED: Data contract violated before poisoning the AI!")
    # In production, this would trigger an alerting system (PagerDuty, Slack, etc
```

8. Storage Formats: CSV vs. Parquet

When storing structured (tabular) data, the format matters immensely for performance.

- CSV (Row-Based Storage):

    - Human-readable plain text.
    - Highly inefficient for AI training workloads. If you want to extract a single feature (e.g., just the "age" column) from a dataset with a million rows, the computer's disk reader must scan horizontally across every single row entirely.
- **Parquet (Columnar Storage):**
    - Machine-optimized, heavily compressed binary format.
    - **The ML Advantage:** Data is stored vertically by column. If your model only needs 5 features out of 100 columns, reading a Parquet file only loads those specific 5 columns into memory, completely bypassing the rest. It is the absolute industry standard for AI data lakes.

9. Handling Sequential & Time-Series Data

When dealing with sequential data (stock prices, clickstreams, sensor logs), time is a strict axis.

The Danger of Random Splits: Using standard randomized split functions (like `train_test_split` in scikit-learn) on sequential data causes **Look-ahead Bias (Data Leakage)**. You are inadvertently giving your model data from December to help it predict outcomes in November during training. The model will look amazing in testing and fail miserably in production.

The Golden Rule: You cannot train an AI to predict the past using information from the future.

The Solution: Temporal Splitting / Rolling Window Validation.

- *Fold 1:* Train on Jan-Mar $\rightarrow$ Predict on April.
- *Fold 2:* Train on Jan-Apr $\rightarrow$ Predict on May.

10. The AI Reproducibility Crisis & DVC

In traditional software, having the right Git commit hash means you can reproduce the exact state of the software. In AI, this is not true.

The Equation of AI State:

$$Model\ State = Code + Data + Environment$$

Code is useless without the exact data it was trained on. Because data constantly updates, if we don't version our datasets alongside our code, our AI systems become entirely non-reproducible. This is a massive compliance and debugging risk.

Why Git Fails for ML: Git tracks code perfectly, but it chokes and crashes on massive 50GB Parquet files or image datasets.

The Solution: Data Version Control (DVC) DVC acts exactly like Git, but for massive datasets. It calculates a unique hash of your massive dataset, stores the heavy data securely in cloud storage (like Amazon S3 or Google Cloud Storage), and leaves a tiny `.dvc` text pointer file in your Git repository.

```
# 1. Track the massive dataset with DVC (hashes the data, creates a pointer)
dvc add data/training_data.parquet

# 2. Track the lightweight DVC pointer with Git
git add data/training_data.parquet.dvc
git commit -m "Added Week 2 training dataset"

# 3. Push the heavy data up to cloud storage (S3/GCS)
dvc push
```

Now, when you checkout an old Git commit and run `dvc pull`, you get the exact dataset as it existed at that moment in time. Your AI is now 100% reproducible.

11. Summary & Next Steps

Key Takeaways:

1. **Architecture:** ELT is the modern standard; choose batch vs. stream processing based on product latency needs.
2. **Math:** Always scale data (Min-Max/Z-Score) to help models converge, and beware of outliers when imputing missing values (Median > Mean).
3. **Storage:** Use Parquet over CSV for blazing-fast columnar feature extraction out of data lakes.
4. **Validation:** Use strict data contracts (Pydantic) to prevent silent AI failures.
5. **Reproducibility:** Version your massive datasets alongside your code using tools like DVC.